

OSSP Skill Week 5

Task 1: Change Directory

Aim

To change the current working directory, validate the given path, maintain the previous directory and handle invalid paths.

Program

```
cd.c
1  cat > cd.c << 'EOF'
2  #include <stdio.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <limits.h>
6
7  int main(void)
8  {
9      char current[PATH_MAX];
10     char previous[PATH_MAX];
11     char path[PATH_MAX];
12
13     if (getcwd(current, sizeof(current)) == NULL) {
14         perror("getcwd");
15         return 1;
16     }
17
18     strcpy(previous, current);
19     printf("Current Directory: %s\n", current);
20
21     printf("Enter directory to change to: ");
22     if (fgets(path, sizeof(path), stdin) == NULL) {
23         fprintf(stderr, "No path entered.\n");
24         return 1;
25     }
26     path[strcspn(path, "\n")] = '\0';
27
28     if (chdir(path) == 0) {
29         printf("Directory changed successfully.\n");
30
31         if (getcwd(current, sizeof(current)) == NULL) {
32             perror("getcwd");
33             return 1;
34         }
35
36         printf("Previous Directory: %s\n", previous);
37         printf("Current Directory: %s\n", current);
38     } else {
39         perror("Error changing directory");
40         return 1;
41     }
42
43     return 0;
44 }
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc cd.c -o cd
./cd
Current Directory: /Users/hrushika/Downloads/pjsdj
Enter directory to change to: documenta
Error changing directory: No such file or directory
hrushika@Hrushikas-MacBook-Air pjsdj % gcc cd.c -o cd
./cd
Current Directory: /Users/hrushika/Downloads/pjsdj
Enter directory to change to: pjsdj
Error changing directory: No such file or directory
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The path was validated by `chdir()`. A valid path changed the working directory and both directories were displayed, while an invalid path produced the error "No such file or directory".

Result

Directory navigation was implemented successfully with previous and current directory tracking.

Task 2: Built-in Command Dispatch

Aim

To identify built-in shell commands and execute them directly inside the shell process using a command dispatch table.

Program

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

static void cmd_pwd(char *arg)
{
    char cwd[1024];
    (void)arg;

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

static void cmd_cd(char *path)
{
    if (path == NULL || strlen(path) == 0)
        printf("cd: missing argument\n");
    else if (chdir(path) != 0)
        perror("cd");
}

static void cmd_exit(char *arg)
{
    (void)arg;
    printf("Exiting shell...\n");
    exit(0);
}

/* Dispatch table: command name -> handler */
struct builtin {
    const char *name;
    void (*handler)(char *arg);
};

static const struct builtin builtins[] = {
    { "pwd", cmd_pwd },
    { "cd", cmd_cd },
    { "exit", cmd_exit },
};

#define NUM_BUILTINS (sizeof(builtins) / sizeof(builtins[0]))

int main(void)
{
    char input[100];

    printf("Built-in Shell\n");

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            cmd_exit(NULL);

        input[strcspn(input, "\n")] = '\0';

        char *name = strtok(input, " \t");
        if (name == NULL)
            continue; /* Empty command */
        char *arg = strtok(NULL, " \t");

        size_t i;
```

```
static const struct builtin builtins[] = {
    { "pwd", cmd_pwd },
    { "exit", cmd_exit },
};

#define NUM_BUILTINS (sizeof(builtins) / sizeof(builtins[0]))

int main(void)
{
    char input[100];

    printf("Built-in Shell\n");

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            cmd_exit(NULL);

        input[strcspn(input, "\n")] = '\0';

        char *name = strtok(input, " \t");
        if (name == NULL)
            continue; /* Empty command */
        char *arg = strtok(NULL, " \t");

        size_t i;
        for (i = 0; i < NUM_BUILTINS; i++) {
            if (strcmp(name, builtins[i].name) == 0) {
                builtins[i].handler(arg);
                break;
            }
        }

        if (i == NUM_BUILTINS)
            printf("Invalid command: %s\n", name);
    }

    return 0;
}
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc builtin.c -o builtin
hrushika@Hrushikas-MacBook-Air pjsdj % ./builtin
Built-in Shell
myShell> hi
Invalid command: hi
myShell> pwd
/Users/hrushika/Downloads/pjsdj
myShell> ls
Invalid command: ls
myShell> ls -l
Invalid command: ls
myShell> mkdir
Invalid command: mkdir
myShell> 
```

Observation

The shell found `pwd`, `cd` and `exit` in the dispatch table and ran them in its own process. The `cd` command changed the directory shown by the next `pwd`, and the unknown command `hello` was reported without ending the shell.

Task 3: `pwd` and `exit`

Aim

To retrieve and display the current working directory and to process the `exit` command safely.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{
    char cwd[1024];
    char command[50];

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            strcpy(command, "exit"); /* Treat end of input as exit */

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "pwd") == 0) {
            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("Current Directory: %s\n", cwd);
            else
                perror("getcwd");
        } else if (strcmp(command, "exit") == 0) {
            printf("Saving shell state...\n");
            printf("Cleaning up resources...\n");
            printf("Shell exited successfully.\n");
            exit(0);
        } else if (strlen(command) == 0) {
            continue;
        } else {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc pwd_exit.c -o pwd_exit
hrushika@Hrushikas-MacBook-Air pjsdj % ./pwd_exit
myShell> hi
Invalid command.
myShell> pwd
Current Directory: /Users/hrushika/Downloads/pjsdj
myShell> hello
Invalid command.
myShell> exit
Saving shell state...
Cleaning up resources...
Shell exited successfully.
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The pwd command retrieved the current working directory using getcwd(). The invalid command was reported, and exit printed the cleanup messages before ending the program.

Result

The current directory was displayed successfully and the shell exited correctly when requested.

Task 4: export Command

Aim

To parse the export command, validate the variable name, create or update the environment variable and verify the stored value.

Program

```
export.c: Command only
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* A valid name starts with a letter or '_' and contains only
   letters, digits and '_' */
static int valid_name(const char *name)
{
    if (name[0] == '\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main(void)
{
    char input[200];

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    if (strncmp(input, "export ", 7) != 0) {
        printf("Invalid command.\n");
        return 1;
    }

    char *assignment = input + 7;
    char *equal = strchr(assignment, '=');

    if (equal == NULL) {
        printf("Syntax Error: Expected NAME=VALUE\n");
        return 1;
    }

    *equal = '\0';
    char *name = assignment;
    char *value = equal + 1;

    if (!valid_name(name)) {
        printf("Invalid variable name.\n");
        return 1;
    }

    if (setenv(name, value, 1) == 0) {
        printf("Environment variable updated successfully.\n");
        printf("%s=%s\n", name, getenv(name));
    } else {
        perror("setenv");
        return 1;
    }

    return 0;
}
```

Output

```
Shell exited successfully.
hrushika@Hrushikas-MacBook-Air pjsdj % gcc export.c -o export
hrushika@Hrushikas-MacBook-Air pjsdj % ./export
myShell> hi
Invalid command.
hrushika@Hrushikas-MacBook-Air pjsdj % ./export
myShell> hello
Invalid command.
hrushika@Hrushikas-MacBook-Air pjsdj % ./export
myShell> pwd
Invalid command.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The export command was split into a name and a value. Valid names were stored with `setenv()` and displayed, while a name starting with a digit and a command without `=` were rejected.

Result

Environment variable creation and updating were implemented and tested successfully.